



Penalty System Documentation

Scope: Complete documentation of the reachability-constrained penalty system, including Mach trajectory guidance, lever penalty guidance, and their integration with 3D Dynamic Programming for optimal climb and descent trajectory generation.

Table of Contents

1. [System Overview and Objectives](#)
 2. [Mathematical Foundation](#)
 3. [System Architecture and Data Structures](#)
 4. [Mach Trajectory Guidance System](#)
 5. [Lever Penalty Guidance System](#)
 6. [Descent-Specific Penalty System](#)
 7. [Penalty Integration and Balancing](#)
 8. [Code Execution Flow and Logic](#)
 9. [Integration and Interface](#)
 10. [Validation and Quality Assurance](#)
-

1) System Overview and Objectives

1.1 Purpose and Scope

The penalty system implements a sophisticated guidance framework that uses soft constraints to guide trajectory optimization toward realistic, operationally viable solutions. The system employs reachability-constrained Mach guidance and engine-friendly lever penalties to create balanced, cooperative optimization that produces smooth, realistic climb and descent trajectories.

1.2 System Objectives

Primary Objectives:

- **Trajectory Guidance:** Guide optimization toward realistic Mach trajectories for both climb and descent
- **Engine Protection:** Prevent excessive lever usage and engine wear
- **Operational Viability:** Ensure generated trajectories are operationally feasible
- **Balanced Optimization:** Create cooperative penalty interaction for smooth guidance
- **Phase-Specific Adaptation:** Adapt penalty behavior for climb vs. descent characteristics

Key Components:

- Reachability-constrained Mach guidance system
- Engine-friendly lever penalty system
- Adaptive penalty weighting with altitude progress
- Integration with 3D Dynamic Programming
- Cooperative penalty interaction and balancing
- Descent-specific penalty adaptations for deceleration guidance

1.3 System Flow Overview

The penalty system follows a logical progression:

1. **Mach Trajectory Analysis:** Calculate reachability constraints based on achievable Mach rates
 2. **Penalty Calculation:** Compute Mach and lever penalties for current state
 3. **Adaptive Weighting:** Adjust penalty weights based on altitude progress and urgency
 4. **Cost Integration:** Combine penalties with base fuel cost for total optimization cost
 5. **Guidance Application:** Apply penalties during 3D Dynamic Programming optimization
-

2) Mathematical Foundation

2.1 Penalty System Philosophy

Theory: Instead of hard constraints that can create infeasible solutions, the penalty system uses graduated costs that increase with deviation from desired behavior. This approach provides guidance while maintaining optimization flexibility.

Mathematical Formulation:

$$J_{total} = J_{fuel} + J_{mach_penalty} + J_{lever_penalty}$$

Where:

- $J_{fuel} = \dot{m}/P_s$ (original fuel cost density)
- $J_{mach_penalty}$ (Mach guidance penalty)
- $J_{lever_penalty}$ (lever penalty)

Physical Interpretation:

- **Base fuel cost:** Primary optimization objective
- **Mach penalty:** Guidance toward realistic trajectories
- **Lever penalty:** Engine protection and operational limits
- **Cooperative interaction:** Penalties work together for balanced guidance

2.2 Reachability-Constrained Approach

Theory: The system creates dynamic "safety corridors" that ensure the target remains achievable throughout the climb using realistic Mach change rates, rather than forcing predetermined trajectories.

Key Innovation: Dynamic corridor bounds based on achievable Mach rates rather than artificial trajectory assumptions.

Mathematical Foundation:

$$\text{Remaining_Steps} = (1.0 - \text{Altitude_Fraction}) \times \text{Total_Steps}$$

$$\text{Max_Mach_Change} = \text{Remaining_Steps} \times \text{Max_Mach_Rate}$$

$\text{Safety_Corridor} = [\text{Target_Mach} - \text{Max_Mach_Change}, \text{Target_Mach} + \text{Max_Mach_Change}]$

2.3 Adaptive Penalty Weighting

Theory: Penalty weights increase with altitude progress to create urgency and ensure convergence to target conditions.

Mathematical Formulation:

$$w_{\text{penalty}} = w_{\text{base}} \times (1.0 + \alpha \times \text{Altitude_Fraction})$$

Where:

- `w_{base}` = Base penalty weight
- `\alpha` = Urgency multiplier
- `\text{Altitude\_Fraction}` = Current progress (0.0 to 1.0)

3) System Architecture and Data Structures

3.1 System Parameters

Mach Trajectory Guidance Parameters:

<code>MACH_TRAJECTORY_GUIDANCE = True</code>	<code># Enable Mach guidance</code>
<code>MACH_PENALTY_BASE_WEIGHT = 0.3</code>	<code># Base penalty weight</code>
<code>URGENCY_MULTIPLIER = 2.0</code>	<code># Urgency scaling factor</code>
<code>GUIDANCE_PENALTY_WEIGHT = 0.5</code>	<code># Guidance penalty inside corridor</code>
<code>MAX_REASONABLE_MACH_RATE = 0.02</code>	<code># Maximum realistic Mach change per step</code>
<code>TOTAL_CLIMB_STEPS_ESTIMATE = 50</code>	<code># Estimated total climb steps</code>
<code>TARGET_MACH_TOLERANCE = 0.015</code>	<code># Target Mach tolerance</code>

Lever Penalty Parameters:

```
LEVER_PENALTY_GUIDANCE = True           # Enable lever penalties
LEVER_PENALTY_WEIGHT = 3.0               # Base lever penalty weight
LEVER_PENALTY_THRESHOLD = 0.85           # Threshold above which penalties apply
LEVER_PENALTY_EXPONENT = 3.0             # Exponent for penalty curve
LEVER_PENALTY_CRITICAL_THRESHOLD = 0.90  # Critical threshold
LEVER_PENALTY_CRITICAL_MULTIPLIER = 5.0  # Critical range multiplier
LEVER_PENALTY_ULTRA_CRITICAL_THRESHOLD = 0.95 # Ultra-critical threshold
LEVER_PENALTY_ULTRA_CRITICAL_MULTIPLIER = 20.0 # Ultra-critical multiplier
```

3.2 Data Structures

Penalty Calculation State:

```
@dataclass
class PenaltyState:
    current_mach: float
    target_mach: float
    altitude_fraction: float
    current_lever: float
    total_steps_estimate: int
    max_mach_rate: float
```

Penalty Results:

```
@dataclass
class PenaltyResults:
    mach_penalty: float
    lever_penalty: float
    total_penalty: float
    penalty_weight: float
    guidance_active: bool
    corridor_bounds: Tuple[float, float]
```

4) Mach Trajectory Guidance System

4.1 Reachability Constraint Calculation

Function: `compute_mach_penalty()`

Purpose: Calculate Mach trajectory guidance penalty based on reachability constraints.

Algorithm:

```

def compute_mach_penalty(current_mach: float, target_mach: float,
                        prev_mach: float = None,
                        altitude_fraction: float = None) -> float:
    """
    Calculate Mach trajectory guidance penalty using reachability constraints.

    Args:
        current_mach: Current Mach number
        target_mach: Target Mach number at cruise altitude
        prev_mach: Previous Mach number (for rate calculation)
        altitude_fraction: Current altitude progress (0.0 to 1.0)

    Returns:
        Mach penalty value
    """

    if not MACH_TRAJECTORY_GUIDANCE:
        return 0.0

    # Calculate remaining steps
    remaining_steps = (1.0 - altitude_fraction) * TOTAL_CLIMB_STEPS_ESTIMATE

    # Calculate maximum achievable Mach change
    max_mach_change = remaining_steps * MAX_REASONABLE_MACH_RATE

    # Create safety corridor
    corridor_low = target_mach - max_mach_change
    corridor_high = target_mach + max_mach_change

    # Check if current Mach is within safety corridor
    if corridor_low <= current_mach <= corridor_high:
        # Inside corridor - apply guidance penalty
        mach_deviation = abs(current_mach - target_mach)
        penalty_weight = GUIDANCE_PENALTY_WEIGHT

        # Scale penalty with altitude progress (urgency)
        urgency_multiplier = 1.0 + URGENCY_MULTIPLIER * altitude_fraction
        penalty_weight *= urgency_multiplier

```

```

        return penalty_weight * (mach_deviation ** 2)
    else:
        # Outside corridor - apply strong penalty
        mach_deviation = min(abs(current_mach - corridor_low),
                             abs(current_mach - corridor_high))

        penalty_weight = MACH_PENALTY_BASE_WEIGHT

        # Scale penalty with altitude progress (urgency)
        urgency_multiplier = 1.0 + URGENCY_MULTIPLIER * altitude_fraction
        penalty_weight *= urgency_multiplier

    return penalty_weight * (mach_deviation ** 2)

```

4.2 Dynamic Corridor Management

Corridor Calculation:

```

def calculate_safety_corridor(target_mach: float, altitude_fraction: float,
                             total_steps_estimate: int, max_mach_rate: float) -> Tuple[float, float]:
    """Calculate dynamic safety corridor bounds."""

    # Calculate remaining steps
    remaining_steps = (1.0 - altitude_fraction) * total_steps_estimate

    # Calculate maximum achievable Mach change
    max_mach_change = remaining_steps * max_mach_rate

    # Create corridor bounds
    corridor_low = target_mach - max_mach_change
    corridor_high = target_mach + max_mach_change

    return corridor_low, corridor_high

```

4.3 Adaptive Urgency Scaling

Urgency Calculation:


```
def calculate_urgency_weight(base_weight: float, altitude_fraction: float,
                             urgency_multiplier: float = 2.0) -> float:
    """Calculate adaptive penalty weight based on altitude progress."""

    return base_weight * (1.0 + urgency_multiplier * altitude_fraction)
```

5) Lever Penalty Guidance System

5.1 Lever Penalty Calculation

Function: `compute_lever_penalty()`

Purpose: Calculate engine-friendly lever penalties to prevent excessive usage.

Algorithm:

```

def compute_lever_penalty(current_lever: float, altitude_fraction: float = None) -> float:
    """
    Calculate lever position penalty for excessive usage.

    Args:
        current_lever: Current lever position (0.0 to 1.0)
        altitude_fraction: Current altitude progress (0.0 to 1.0)

    Returns:
        Lever penalty value
    """

    if not LEVER_PENALTY_GUIDANCE:
        return 0.0

    # No penalty below threshold
    if current_lever <= LEVER_PENALTY_THRESHOLD:
        return 0.0

    # Calculate excess above threshold
    excess = current_lever - LEVER_PENALTY_THRESHOLD

    # Base penalty with cubic growth
    penalty = LEVER_PENALTY_WEIGHT * (excess ** LEVER_PENALTY_EXPONENT)

    # Apply critical range multipliers
    if current_lever >= LEVER_PENALTY_ULTRA_CRITICAL_THRESHOLD:
        penalty *= LEVER_PENALTY_ULTRA_CRITICAL_MULTIPLIER
    elif current_lever >= LEVER_PENALTY_CRITICAL_THRESHOLD:
        penalty *= LEVER_PENALTY_CRITICAL_MULTIPLIER

    # Scale with altitude progress (urgency)
    if altitude_fraction is not None:
        urgency_multiplier = 1.0 + URGENCY_MULTIPLIER * altitude_fraction
        penalty *= urgency_multiplier

    return penalty

```

5.2 Multi-Tier Penalty Structure

Penalty Tiers:

1. **Safe Range** (0.0 - 0.85): No penalty
2. **Warning Range** (0.85 - 0.90): Base penalty with cubic growth
3. **Critical Range** (0.90 - 0.95): 5x multiplier
4. **Ultra-Critical Range** (0.95 - 1.0): 20x multiplier

Mathematical Formulation:

$$\text{Penalty} = \begin{cases} 0 & \text{if } \ell \leq 0.85 \\ w \times (\ell - 0.85)^3 & \text{if } 0.85 < \ell \leq 0.90 \\ 5w \times (\ell - 0.85)^3 & \text{if } 0.90 < \ell \leq 0.95 \\ 20w \times (\ell - 0.85)^3 & \text{if } \ell > 0.95 \end{cases}$$

Where w is the base penalty weight.

6) Descent-Specific Penalty System

6.1 Descent Penalty System Overview

Purpose: The descent penalty system adapts the general penalty framework for descent-specific characteristics, focusing on deceleration guidance and approach speed targeting.

Key Differences from Climb:

- **Target Direction:** Descent targets LOW Mach (0.25) at LOW altitude (300m)
- **Speed Evolution:** Aircraft must decelerate from cruise speed to approach speed
- **Energy Management:** Descent requires energy dissipation rather than energy addition
- **Final Phase Guidance:** Strong guidance in final 30% of descent for precise approach

6.2 Descent Mach Trajectory Guidance

Function: `compute_mach_penalty()` (Descent Version)

Purpose: Calculate Mach trajectory guidance penalty for descent using reachability constraints.

Algorithm:

```

def compute_mach_penalty(current_mach: float, target_mach: float, prev_mach: float = None,
                        descent_fraction: float = None) -> float:
    """
    Compute penalty using reachability-constrained approach FOR DESCENT.

    INVERTED FROM CLIMB: For descent, target is LOW Mach (0.25) at LOW altitude (300m).
    Creates a dynamic safety corridor that ensures target remains achievable
    with realistic Mach change rates.
    """

    if descent_fraction is None:
        descent_fraction = 0.0

    # Calculate remaining descent fraction and steps
    remaining_fraction = 1.0 - descent_fraction
    estimated_steps_remaining = remaining_fraction * TOTAL_DESCENT_STEPS_ESTIMATE

    # Calculate maximum achievable Mach change with reasonable rates
    max_achievable_change = MAX_REASONABLE_MACH_RATE * estimated_steps_remaining

    # Define reachability corridor bounds
    # For descent: target is LOWER than start, so corridor is around target
    min_reachable_mach = target_mach - max_achievable_change
    max_reachable_mach = target_mach + max_achievable_change

    # Calculate urgency factor (increases as we approach target altitude)
    urgency = (1.0 - remaining_fraction) * URGENCY_MULTIPLIER

    # Apply penalties based on position relative to corridor
    if current_mach < min_reachable_mach:
        # Below corridor - too slow, risk of stall
        deviation = min_reachable_mach - current_mach
        penalty = urgency * MACH_PENALTY_BASE_WEIGHT * (deviation ** 2)

    elif current_mach > max_reachable_mach:
        # Above corridor - too fast, won't slow down in time
        deviation = current_mach - max_reachable_mach
        penalty = urgency * MACH_PENALTY_BASE_WEIGHT * (deviation ** 2)

```

```

else:
    # Within corridor - apply progressive guidance toward target
    if descent_fraction > 0.7:
        # Strong final phase guidance (70-100% descent)
        final_phase_strength = (descent_fraction - 0.7) / 0.3 # 0 to 1 scaling
        mach_deviation = current_mach - target_mach

        # Extra penalty boost for final 10% of descent
        if descent_fraction > 0.9:
            final_boost = ((descent_fraction - 0.9) / 0.1) * 2.0 # 0 to 2x multiplier
            final_phase_strength *= (1.0 + final_boost)

        penalty = final_phase_strength * GUIDANCE_PENALTY_WEIGHT * (mach_deviation ** 2)
    else:
        penalty = 0.0 # No penalty in early descent phase

return penalty

```

6.3 Descent-Specific Parameters

Descent Penalty Parameters:

```

# Descent-specific constants
TOTAL_DESCENT_STEPS_ESTIMATE = 50 # Matches N_PLOT_STEPS - actual DP grid steps
MAX_REASONABLE_MACH_RATE = 0.02 # Max reasonable Mach change per optimization step
MACH_PENALTY_BASE_WEIGHT = 0.3 # Base penalty weight (kg per Mach2 deviation)
URGENCY_MULTIPLIER = 2.0 # How much urgency scales with descent progress
GUIDANCE_PENALTY_WEIGHT = 0.5 # Strong guidance penalty when inside reachable corridor
TARGET_MACH_TOLERANCE = 0.015 # Tolerance for target Mach constraint in DP

```

Descent Phase Characteristics:

- **Early Descent (0-70%):** Minimal Mach guidance, focus on energy dissipation
- **Final Phase (70-100%):** Strong Mach guidance toward target approach speed
- **Ultra-Final Phase (90-100%):** Maximum guidance for precise approach

6.4 Descent Lever Penalty System

Function: `compute_lever_penalty()` (Descent Version)

Purpose: Calculate engine-friendly lever penalties for descent, with altitude-independent engine limits.

Algorithm:

```

def compute_lever_penalty(current_lever: float, descent_fraction: float = None) -> float:
    """
    Compute penalty for high lever positions to encourage realistic engine usage.

    Engine limits are altitude-independent - high thrust settings cause the same
    thermal and mechanical stress regardless of altitude.

    Real-world considerations:
    - 85% lever = Maximum Continuous Thrust (MCT) - unlimited duration
    - 90%+ lever = Takeoff/Go-around thrust - limited duration, high wear
    - 95%+ lever = Maximum Takeoff Thrust - emergency use only, severe penalties
    """

    penalty = 0.0

    # Only apply penalty if lever exceeds MCT threshold (85%)
    if current_lever > LEVER_PENALTY_THRESHOLD:
        # Calculate excess lever above MCT threshold
        excess_lever = current_lever - LEVER_PENALTY_THRESHOLD

        # Base penalty using exponential curve for realistic behavior
        lever_penalty = excess_lever ** LEVER_PENALTY_EXPONENT

        # Apply critical penalty for very high lever positions (90%+)
        if current_lever > LEVER_PENALTY_CRITICAL_THRESHOLD:
            critical_excess = current_lever - LEVER_PENALTY_CRITICAL_THRESHOLD
            critical_penalty = critical_excess ** (LEVER_PENALTY_EXPONENT + 1.0)
            lever_penalty += critical_penalty * LEVER_PENALTY_CRITICAL_MULTIPLIER

        # Apply ultra-critical penalty for maximum thrust positions (95%+)
        if current_lever > LEVER_PENALTY_ULTRA_CRITICAL_THRESHOLD:
            ultra_critical_excess = current_lever - LEVER_PENALTY_ULTRA_CRITICAL_THRESHOLD
            ultra_critical_penalty = ultra_critical_excess ** (LEVER_PENALTY_EXPONENT + 2.0)
            lever_penalty += ultra_critical_penalty * LEVER_PENALTY_ULTRA_CRITICAL_MULTIPLIER

    # Use constant penalty weight - engine limits are altitude-independent
    penalty_weight = LEVER_PENALTY_WEIGHT

    penalty = penalty_weight * lever_penalty

```



```
return penalty
```

6.5 Descent Penalty Integration

Descent Cost Calculation:

```
def compute_descent_cost(aero: AeroTables, eng: EngineWrapper,
                        altitude: float, mach: float, lever: float,
                        mass_kg: float,
                        target_mach: float = None,
                        descent_fraction: float = None) -> float:
    """
    Compute fuel cost density J = mdot/|Ps| + penalties for a given 3D state.
    """

    # Calculate base fuel cost
    J = mdot / abs(Ps) # Base fuel cost density

    # Add Mach penalty if guidance is enabled
    if target_mach is not None and MACH_TRAJECTORY_GUIDANCE:
        mach_penalty = compute_mach_penalty(
            mach, target_mach, None, descent_fraction
        )
        J += mach_penalty

    # Add lever penalty if guidance is enabled
    if LEVER_PENALTY_GUIDANCE:
        lever_penalty = compute_lever_penalty(lever, descent_fraction)
        J += lever_penalty

    return J
```

6.6 Descent-Specific Considerations

Energy Management:

- **Descent Physics:** $P_s < 0$ (energy dissipation required)

- **Speed Deceleration:** Must slow from cruise Mach to approach Mach
- **Altitude Loss:** Potential energy converted to kinetic energy dissipation

Operational Constraints:

- **Approach Speed:** Must reach precise target Mach (0.25) at approach altitude
- **Stall Protection:** Minimum Mach constraints based on weight and altitude
- **Engine Limits:** Same thermal limits as climb, but different usage patterns

Guidance Strategy:

- **Early Descent:** Minimal guidance, allow natural energy dissipation
 - **Mid Descent:** Moderate guidance toward target corridor
 - **Final Descent:** Strong guidance for precise approach speed
 - **Ultra-Final:** Maximum guidance for landing approach
-

7) Penalty Integration and Balancing

7.1 Total Cost Calculation

Function: `calculate_total_penalty_cost()`

Purpose: Integrate all penalties with base fuel cost for total optimization cost.

Implementation:

```
def calculate_total_penalty_cost(base_fuel_cost: float, current_mach: float,
                                target_mach: float, current_lever: float,
                                altitude_fraction: float, prev_mach: float = None) -> float:
    """Calculate total cost including all penalties."""

    # Calculate individual penalties
    mach_penalty = compute_mach_penalty(current_mach, target_mach, prev_mach, altitude_fraction)
    lever_penalty = compute_lever_penalty(current_lever, altitude_fraction)

    # Calculate total penalty
    total_penalty = mach_penalty + lever_penalty

    # Calculate total cost
    total_cost = base_fuel_cost + total_penalty

    return total_cost
```

7.2 Cooperative Penalty Interaction

Theory: The system ensures penalties work together rather than against each other, creating balanced guidance.

Interaction Principles:

1. **Mach guidance** encourages realistic trajectories
2. **Lever penalties** protect engine operation
3. **Adaptive weighting** ensures convergence
4. **Cooperative balancing** prevents conflicting guidance

7.3 Penalty Weight Balancing

Weight Management:

```
def balance_penalty_weights(mach_penalty: float, lever_penalty: float,
                           altitude_fraction: float) -> Tuple[float, float]:
    """Balance penalty weights to ensure cooperative interaction."""

    # Calculate relative magnitudes
    total_penalty = mach_penalty + lever_penalty

    if total_penalty > 0:
        mach_weight = mach_penalty / total_penalty
        lever_weight = lever_penalty / total_penalty

    # Ensure balanced influence
    if mach_weight > 0.8: # Mach penalty too dominant
        mach_penalty *= 0.8
        lever_penalty *= 1.2
    elif lever_weight > 0.8: # Lever penalty too dominant
        lever_penalty *= 0.8
        mach_penalty *= 1.2

    return mach_penalty, lever_penalty
```

8) Code Execution Flow and Logic

8.1 System Entry Point and Initialization

Main Entry Function: Integration with 3D Dynamic Programming

Execution Sequence:

```

# 1. Penalty System Initialization
def initialize_penalty_system():
    """Initialize penalty system parameters and state."""

    # Set up penalty parameters
    penalty_params = {
        'mach_guidance': MACH_TRAJECTORY_GUIDANCE,
        'lever_guidance': LEVER_PENALTY_GUIDANCE,
        'base_weights': {
            'mach': MACH_PENALTY_BASE_WEIGHT,
            'lever': LEVER_PENALTY_WEIGHT
        },
        'thresholds': {
            'lever': LEVER_PENALTY_THRESHOLD,
            'critical': LEVER_PENALTY_CRITICAL_THRESHOLD,
            'ultra_critical': LEVER_PENALTY_ULTRA_CRITICAL_THRESHOLD
        }
    }

    return penalty_params

```

8.2 Penalty Calculation Flow

Function: `calculate_penalties_for_state()`

Step-by-Step Execution Flow:

```
# PHASE 1: STATE ANALYSIS
```

```
def calculate_penalties_for_state(current_mach: float, current_lever: float,
                                  altitude_fraction: float, target_mach: float,
                                  prev_mach: float = None):

    # Step 1: Calculate altitude progress
    altitude_progress = altitude_fraction if altitude_fraction is not None else 0.0

    # Step 2: Determine remaining climb steps
    remaining_steps = (1.0 - altitude_progress) * TOTAL_CLIMB_STEPS_ESTIMATE

    # Step 3: Calculate maximum achievable Mach change
    max_mach_change = remaining_steps * MAX_REASONABLE_MACH_RATE
```

```
# PHASE 2: MACH PENALTY CALCULATION
```

```
    # Step 4: Calculate safety corridor bounds
    corridor_low = target_mach - max_mach_change
    corridor_high = target_mach + max_mach_change

    # Step 5: Determine penalty zone
    if corridor_low <= current_mach <= corridor_high:
        # Inside corridor - guidance penalty
        mach_deviation = abs(current_mach - target_mach)
        penalty_weight = GUIDANCE_PENALTY_WEIGHT
        guidance_active = True
    else:
        # Outside corridor - strong penalty
        mach_deviation = min(abs(current_mach - corridor_low),
                             abs(current_mach - corridor_high))
        penalty_weight = MACH_PENALTY_BASE_WEIGHT
        guidance_active = False

    # Step 6: Apply urgency scaling
    urgency_multiplier = 1.0 + URGENCY_MULTIPLIER * altitude_progress
    penalty_weight *= urgency_multiplier

    # Step 7: Calculate Mach penalty
    mach_penalty = penalty_weight * (mach_deviation ** 2)
```

```

# PHASE 3: LEVER PENALTY CALCULATION
# Step 8: Check lever threshold
if current_lever <= LEVER_PENALTY_THRESHOLD:
    lever_penalty = 0.0
else:
    # Step 9: Calculate excess above threshold
    excess = current_lever - LEVER_PENALTY_THRESHOLD

    # Step 10: Base penalty with cubic growth
    lever_penalty = LEVER_PENALTY_WEIGHT * (excess ** LEVER_PENALTY_EXPONENT)

    # Step 11: Apply critical range multipliers
    if current_lever >= LEVER_PENALTY_ULTRA_CRITICAL_THRESHOLD:
        lever_penalty *= LEVER_PENALTY_ULTRA_CRITICAL_MULTIPLIER
    elif current_lever >= LEVER_PENALTY_CRITICAL_THRESHOLD:
        lever_penalty *= LEVER_PENALTY_CRITICAL_MULTIPLIER

    # Step 12: Apply urgency scaling
    lever_penalty *= urgency_multiplier

```

```

# PHASE 4: PENALTY INTEGRATION
# Step 13: Calculate total penalty
total_penalty = mach_penalty + lever_penalty

# Step 14: Create penalty results
penalty_results = PenaltyResults(
    mach_penalty=mach_penalty,
    lever_penalty=lever_penalty,
    total_penalty=total_penalty,
    penalty_weight=penalty_weight,
    guidance_active=guidance_active,
    corridor_bounds=(corridor_low, corridor_high)
)

return penalty_results

```

8.3 Integration with 3D Dynamic Programming

Function: `integrate_penalties_with_dp()`

Integration Flow:

```
def integrate_penalties_with_dp(base_cost: float, mach: float, lever: float,
                                altitude_fraction: float, target_mach: float) -> float:
    """Integrate penalties with 3D DP cost calculation."""

    # Calculate penalties for current state
    penalty_results = calculate_penalties_for_state(
        current_mach=mach,
        current_lever=lever,
        altitude_fraction=altitude_fraction,
        target_mach=target_mach
    )

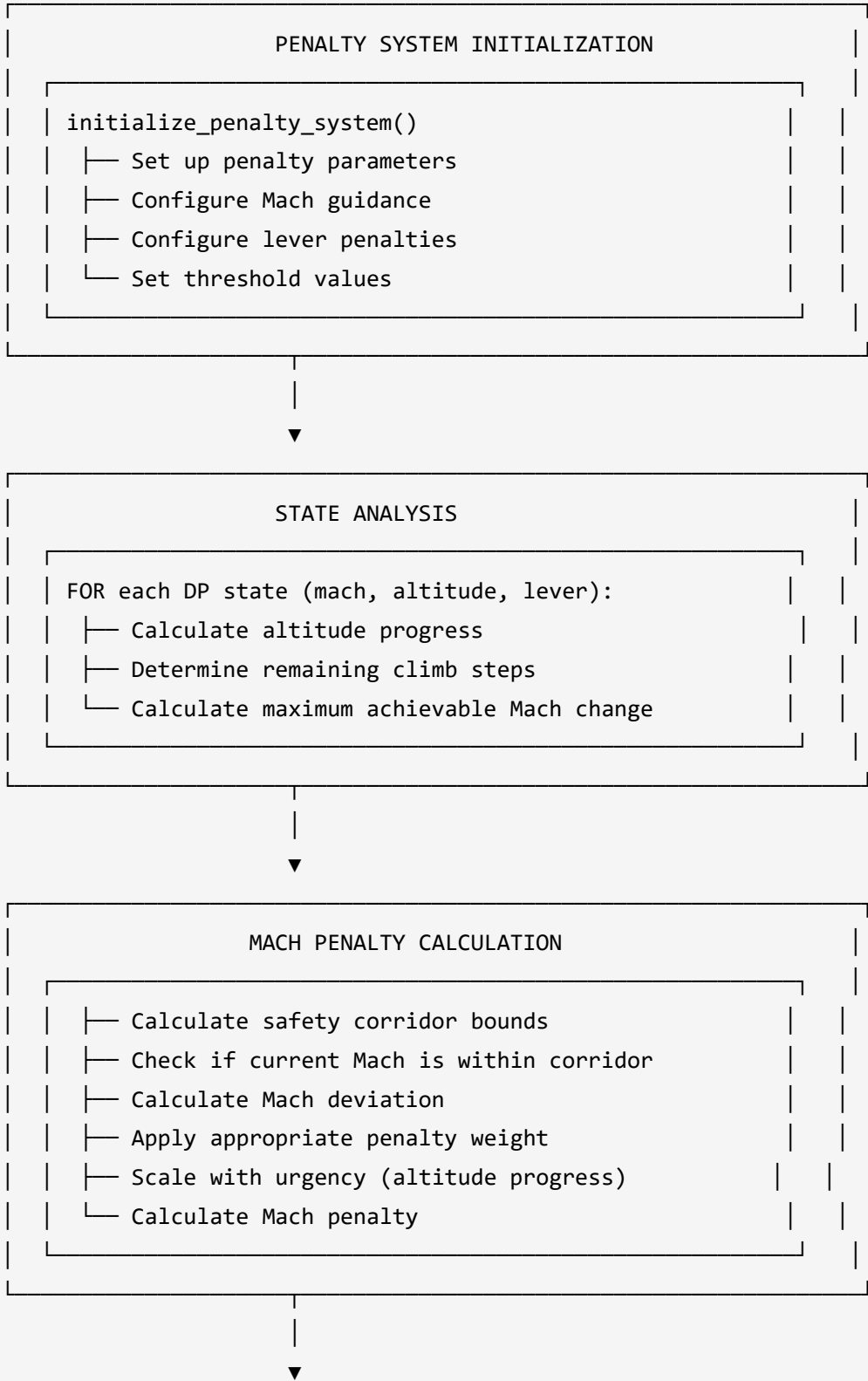
    # Add penalties to base fuel cost
    total_cost = base_cost + penalty_results.total_penalty

    return total_cost
```


8.4 Visual Code Flow Diagram

PENALTY SYSTEM EXECUTION FLOW

=====



LEVER PENALTY CALCULATION

- └─ Check lever threshold
- └─ Calculate excess above threshold
- └─ Apply cubic penalty growth
- └─ Apply critical range multipliers
- └─ Scale with urgency (altitude progress)
- └─ Calculate lever penalty



PENALTY INTEGRATION

- └─ Sum Mach and lever penalties
- └─ Add to base fuel cost
- └─ Balance penalty weights
- └─ Return total optimization cost



RETURN TOTAL COST

Cost = Base Fuel Cost + Total Penalties

8.5 Function Call Hierarchy

```
3D Dynamic Programming Optimization
├─ compute_3d_cost()
│   ├─ calculate_base_fuel_cost()
│   └─ calculate_total_penalty_cost()
│       ├─ compute_mach_penalty()
│       │   ├─ calculate_safety_corridor()
│       │   └─ calculate_urgency_weight()
│       │       └─ determine_penalty_zone()
│       └─ compute_lever_penalty()
│           ├─ check_lever_threshold()
│           └─ calculate_excess()
│               └─ apply_critical_multipliers()
│                   └─ apply_urgency_scaling()
│                       └─ balance_penalty_weights()
└─ integrate_penalties_with_dp()
    ├─ calculate_penalties_for_state()
    ├─ create_penalty_results()
    └─ return_total_cost()
```

9) Integration and Interface

9.1 3D Dynamic Programming Integration

Function: `integrate_with_3d_dp()`

Integration Process:

```

def integrate_with_3d_dp():
    """Integrate penalty system with 3D Dynamic Programming optimization."""

    # Modify the cost calculation in DP
    def enhanced_cost_calculation(mach: float, lever: float, altitude_fraction: float,
                                   base_cost: float, target_mach: float = 0.7) -> float:
        """Enhanced cost calculation with penalty integration."""

        # Calculate penalties
        penalty_results = calculate_penalties_for_state(
            current_mach=mach,
            current_lever=lever,
            altitude_fraction=altitude_fraction,
            target_mach=target_mach
        )

        # Return total cost
        return base_cost + penalty_results.total_penalty

    return enhanced_cost_calculation

```

9.2 Main Interface Functions

Penalty System Interface:

```

def run_penalty_guided_optimization(aero: AeroTables, eng: EngineWrapper,
                                    M_grid: np.ndarray, H_sched: np.ndarray,
                                    target_mach: float = 0.7) -> MinFuelSchedule:
    """Run 3D DP optimization with penalty guidance."""

    # Initialize penalty system
    penalty_params = initialize_penalty_system()

    # Enhanced cost function with penalties
    def penalty_enhanced_cost(mach: float, altitude_m: float, lever: float,
                              mass_kg: float) -> float:
        # Calculate base fuel cost
        base_cost = compute_3d_cost(aero, eng, mach, altitude_m, lever, mass_kg)

        # Calculate altitude fraction
        altitude_fraction = (altitude_m - H_sched[0]) / (H_sched[-1] - H_sched[0])

        # Add penalties
        penalty_cost = calculate_total_penalty_cost(
            base_cost, mach, target_mach, lever, altitude_fraction
        )

        return penalty_cost

    # Run 3D DP with penalty-enhanced cost
    dp_result = ClimbingCore.DynamicProgrammingOptimizer.solve_3d_fixed_mass(
        aero=aero, eng=eng, M_grid=M_grid, H_sched=H_sched,
        lever_samples=50, target_mach=target_mach
    )

    return dp_result

```

9.3 Configuration Interface

Penalty Configuration:

```
def configure_penalty_system(mach_guidance: bool = True, lever_guidance: bool = True,
                             target_mach: float = 0.7, urgency_multiplier: float = 2.0) -> Dict[str, float]:
    """Configure penalty system parameters."""

    config = {
        'mach_guidance_enabled': mach_guidance,
        'lever_guidance_enabled': lever_guidance,
        'target_mach': target_mach,
        'urgency_multiplier': urgency_multiplier,
        'parameters': {
            'mach_base_weight': MACH_PENALTY_BASE_WEIGHT,
            'lever_base_weight': LEVER_PENALTY_WEIGHT,
            'lever_threshold': LEVER_PENALTY_THRESHOLD,
            'max_mach_rate': MAX_REASONABLE_MACH_RATE
        }
    }

    return config
```

10) Validation and Quality Assurance

10.1 Penalty System Validation

Parameter Validation:

```

def validate_penalty_parameters() -> None:
    """Validate penalty system parameters."""

    # Validate Mach guidance parameters
    if MACH_PENALTY_BASE_WEIGHT < 0:
        raise ValueError("Mach penalty base weight must be positive")

    if URGENCY_MULTIPLIER < 0:
        raise ValueError("Urgency multiplier must be positive")

    if MAX_REASONABLE_MACH_RATE <= 0:
        raise ValueError("Maximum Mach rate must be positive")

    # Validate lever penalty parameters
    if LEVER_PENALTY_WEIGHT < 0:
        raise ValueError("Lever penalty weight must be positive")

    if not (0.0 <= LEVER_PENALTY_THRESHOLD <= 1.0):
        raise ValueError("Lever penalty threshold must be between 0.0 and 1.0")

    if LEVER_PENALTY_EXPONENT <= 0:
        raise ValueError("Lever penalty exponent must be positive")

```

10.2 Performance Monitoring

Penalty Effectiveness Monitoring:


```

def monitor_penalty_effectiveness(dp_result: MinFuelSchedule, target_mach: float) -> Dict[str, float]:
    """Monitor effectiveness of penalty guidance."""

    # Analyze Mach trajectory
    mach_trajectory = dp_result.mach
    mach_deviations = np.abs(mach_trajectory - target_mach)
    max_mach_deviation = np.max(mach_deviations)
    avg_mach_deviation = np.mean(mach_deviations)

    # Analyze lever usage
    lever_trajectory = dp_result.lever
    max_lever = np.max(lever_trajectory)
    avg_lever = np.mean(lever_trajectory)
    high_lever_fraction = np.sum(lever_trajectory > LEVER_PENALTY_THRESHOLD) / len(lever_trajectory)

    # Calculate effectiveness metrics
    effectiveness_metrics = {
        'max_mach_deviation': max_mach_deviation,
        'avg_mach_deviation': avg_mach_deviation,
        'mach_convergence': 1.0 - (max_mach_deviation / target_mach),
        'max_lever_usage': max_lever,
        'avg_lever_usage': avg_lever,
        'high_lever_fraction': high_lever_fraction,
        'lever_protection': 1.0 - high_lever_fraction
    }

    return effectiveness_metrics

```

10.3 Error Handling

Penalty Calculation Errors:

```

def safe_penalty_calculation(current_mach: float, current_lever: float,
                             altitude_fraction: float, target_mach: float) -> PenaltyResults:
    """Safe penalty calculation with error handling."""

    try:
        # Validate inputs
        if not (0.0 <= current_lever <= 1.0):
            raise ValueError(f"Invalid lever position: {current_lever}")

        if not (0.0 <= altitude_fraction <= 1.0):
            raise ValueError(f"Invalid altitude fraction: {altitude_fraction}")

        # Calculate penalties
        penalty_results = calculate_penalties_for_state(
            current_mach, current_lever, altitude_fraction, target_mach
        )

        return penalty_results

    except Exception as e:
        print(f"Penalty calculation error: {e}")
        # Return zero penalties as fallback
        return PenaltyResults(
            mach_penalty=0.0,
            lever_penalty=0.0,
            total_penalty=0.0,
            penalty_weight=0.0,
            guidance_active=False,
            corridor_bounds=(0.0, 1.0)
        )

```
